

$\Rightarrow$ `pid_t waitpid(pid_t pid, int *status, int options);`

① `pid_t pid`: 指定要等待的子进程
 ② `int *status`: 为子进程ID.
 ③ `int options`: 等待与调用进程同进程组的任何子进程.

`-1`: 子进程

`code/ecf/waitpid2.c`

```

1  #include "csapp.h"
2  #define N 2
3
4  int main()
5  {
6      int status, i;
7      pid_t pid[N], retpid;
8
9      /* Parent creates N children */
10     for (i = 0; i < N; i++)
11         if ((pid[i] = Fork()) == 0) /* Child */
12             exit(100+i);
13
14     /* Parent reaps N children in order */
15     i = 0;
16     while ((retpid = waitpid(pid[i++], &status, 0)) > 0) {
17         if (WIFEXITED(status))
18             printf("child %d terminated normally with exit status=%d\n",
19                   retpid, WEXITSTATUS(status));
20         else
21             printf("child %d terminated abnormally\n", retpid);
22     }
23
24     /* The only normal termination is if there are no more children */
25     if (errno != ECHILD)
26         unix_error("waitpid error");
27
28     exit(0);
29 }
  
```

① `pid_t pid`: 指定要等待的子进程
 ② `int *status`: 保存子进程的退出状态
 ③ `int options`: 调用 `Waitpid` 的行为

`WNONBLOCK`: 将阻塞, 未就绪时立即返回 0
`WUNTRACED`: 会返回因信号杀死的子进程
`WCONTINUED`: 返回从暂停中恢复的子进程

`code/ecf/waitpid2.c`

图 8-19 使用 `waitpid` 按照创建子进程的顺序来回收这些僵死子进程

练习题 8.4 考虑下面的程序:

`code/ecf/waitprobl.c`

```

1  int main()
2  {
3      int status;
4      pid_t pid;
5
6      printf("Hello\n");
7      pid = Fork();
8      printf("%d\n", !pid);
9      if (pid != 0) {
10         if (waitpid(-1, &status, 0) > 0) {
11             if (WIFEXITED(status) != 0)
12                 printf("%d\n", WEXITSTATUS(status));
13         }
14     }
15     printf("Bye\n");
16     exit(2);
17 }
  
```

`code/ecf/waitprobl.c`